

Project TicketDesk

Capstone POC Brief - Deploy a Real Application on AWS (Foundation Level)

July 2026

Table of Contents

Project TicketDesk

Capstone POC Brief — Deploy a Real Application on AWS

Level: Foundation (freshers) | **Duration:** 2 weeks (10 working days) **Streams:** Java · .NET · Python · Testing **Goal:** By the end of this POC, you can take a working application and deploy it properly on AWS — on your own, from code, repeatably.

1. What this POC is really about

Let's be clear about the objective, because it changes how you should spend your time.

You are not being assessed on writing application code. The application is given to you, already working, in Java, .NET and Python. It is a small IT support ticket tracker — nothing clever.

You are being assessed on getting it onto AWS properly. Containerised. Behind a load balancer. Database in a private subnet. Password in a secrets store, not in a config file. Deployed by a pipeline, not by you clicking around. Logs where you can find them. Alarms that tell you when something breaks. And the whole thing rebuildable from zero with one command.

That skill — “I can deploy and run an application on AWS” — is the single most in-demand thing a fresher can walk onto a project with. Most people finish cloud training able to *describe* it. Very few can *do* it. In two weeks, you will be able to do it.

2. The application: TicketDesk

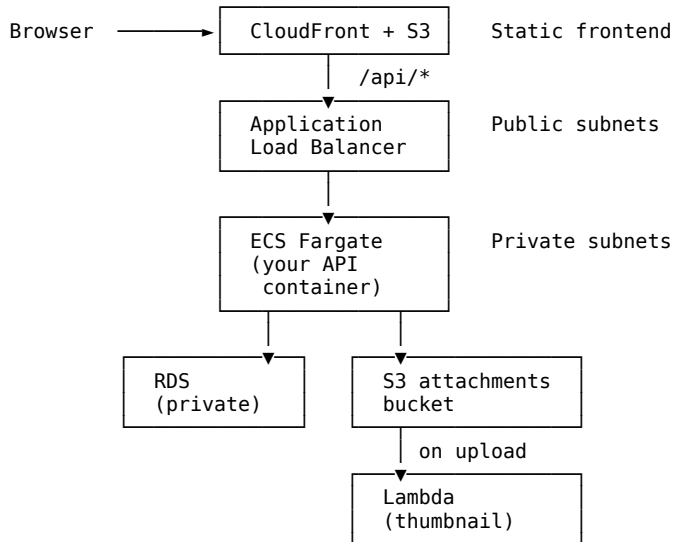
An IT support ticket tracker. You have raised tickets like these yourself.

Feature	What it does
Create a ticket	Title, description, category, priority
List and filter tickets	By status, priority, category
Update status	OPEN → IN_PROGRESS → RESOLVED → CLOSED
Add a comment	Simple threaded notes on a ticket
Attach a file	One screenshot or document per ticket
Dashboard	Counts by status and priority

All of this code is provided. You will find it in the starter repository in three flavours — Spring Boot, ASP.NET Core, and FastAPI. Pick the one matching your stream.

You may need to make **small** code changes: reading a config value from an environment variable, adding a health endpoint, generating a presigned URL. Those are part of the learning. You should not be writing features.

3. What you will build on AWS



Supporting: Secrets Manager · Parameter Store · ECR · CloudWatch
logs/metrics/alarms · IAM roles · Terraform · CI/CD pipeline

Nine AWS services. That is deliberately a small list. You will know all nine properly rather than twenty vaguely.

4. The eight milestones

This is the heart of the POC. Each milestone is small, independently verifiable, and signed off before you move on. **Do not skip ahead** — each one depends on the last.

PART 1 — Individual (Days 1–6)

You do these **yourself**, in your own resource prefix (tkt-<your-initials>-*) inside your pod's AWS account. Everyone deploys their own copy. This is how we know *you* can do it, not that somebody in your pod can.

M0 — Deploy it by hand (Day 1)

Deploy the API to ECS Fargate behind a load balancer using **only the AWS console**, following the guided runbook provided.

Yes, by hand — on purpose. You need to see the pieces and feel how many clicks it takes before you understand why nobody does it this way. At the end of the day, **delete everything you created**.

Done when: you can hit the load balancer URL and get a response, and you have written down every resource you had to create.

M1 — Containerise (Day 2)

Write a Dockerfile for your API. Build it, run it locally, push it to ECR.

Requirements: multi-stage build, runs as a **non-root user**, no build tools left in the final image, and a tag based on the **git commit SHA** — not latest.

Done when: your image is in ECR with a traceable tag, and `docker run` works locally.

M2 — Infrastructure as code (Day 3)

Now build M0 again — but entirely in Terraform. VPC, two public subnets, two private subnets, security groups, load balancer, target group, ECS cluster, task definition, service.

The container runs in a **private** subnet. The load balancer is the only thing in a public one.

Done when: terraform apply from nothing produces a working URL, and terraform destroy removes it cleanly.

M3 — Database and secrets (Day 4)

Add RDS (PostgreSQL or MySQL) in a private subnet, not publicly accessible. The database password lives in **Secrets Manager**. Other config lives in **Parameter Store**. Your task pulls both at runtime using its IAM task role.

Run the provided schema migration on deploy.

Done when: you create a ticket in the app, restart the container, and the ticket is still there. And there is **no password anywhere in your repository**.

M4 — Frontend (Day 5)

Deploy the provided static frontend to S3, served through CloudFront, with `/api/*` routed to your load balancer. The S3 bucket itself must **not** be public.

Done when: you open the CloudFront URL and the full app works end to end.

M5 — Your first serverless piece (Day 6)

File attachments. Your API issues a **presigned S3 URL** so the browser uploads directly to S3. When the file lands, an S3 event triggers a **Lambda function** that generates a thumbnail and writes it back to a different prefix.

Done when: you attach a screenshot to a ticket, and a thumbnail appears without your API ever touching the file bytes.

End of Day 6: individual sign-off. You demo your own deployment to a facilitator, 10 minutes.

PART 2 — Pod of 3-4 (Days 7-10)

Your pod now picks **one** member's deployment as the shared reference and builds the professional layer on top of it together.

M6 — CI/CD pipeline (Day 7)

No more running deploy commands by hand. Build a pipeline (GitHub Actions or CodePipeline) that on every push to main: builds the image → runs unit tests → scans for committed secrets → pushes to ECR → updates the ECS service → runs a smoke test against the deployed URL.

A failing step must **block** the deployment.

Done when: you change one line of text in the app, push, and watch it reach production without touching AWS.

M7 — Observability (Day 8)

- Application logs in CloudWatch, with a **retention period set** (not “never expire”)
- A dashboard showing: request count, error rate, response time, CPU/memory, database connections
- **Three alarms** that actually notify someone: 5xx errors, unhealthy targets, high database CPU

Done when: a facilitator breaks your app and you find out from an alarm, not from them telling you.

M8 — Harden and prove it (Days 9–10)

- Fix everything on the Deployment Readiness Checklist (§6) that is still failing
- Tag every resource: Project, Owner, Environment, CostCenter
- Run the smoke test suite and a **light** load sanity check (20 concurrent users, 5 minutes — we are looking for “no errors”, not percentile targets)
- Produce a one-page cost report: what you spent, on what, and the two most expensive things
- **Destroy the entire stack and rebuild it from zero.** This is the real test.

Done when: your pod runs `terraform destroy`, then `terraform apply`, and the app comes back working — while a facilitator watches.

5. Who does what in the pod

Stream	Individual work (M0–M5)	Pod focus (M6–M8)
Java	Deploy the Spring Boot API	Pipeline build stage, container tuning, health endpoints
.NET	Deploy the ASP.NET Core API	Pipeline, config handling, graceful shutdown
Python	Deploy the FastAPI service	The Lambda function, presigned uploads, deploy scripts
Testing	Deploy the FastAPI service (<i>you deploy too — no exceptions</i>)	Smoke tests in the pipeline, load sanity check, verifying the checklist, breaking things on purpose

Testing stream, note: you are doing the full deployment track like everyone else. A tester who cannot deploy the thing they are testing is at a permanent disadvantage on cloud projects. Your additional pod deliverables are the smoke test suite and the checklist verification.

6. Deployment Readiness Checklist

This is the checklist we will walk through on demo day. It is also your daily to-do list — print it and tick items off. **It is not a surprise; there are no hidden criteria.**

Container

- ☐ 1. Multi-stage Dockerfile
- ☐ 2. Container runs as a non-root user
- ☐ 3. No SDK, compiler or build tools in the final image
- ☐ 4. Image tagged with the git commit SHA, not latest
- ☐ 5. Image scanning enabled on the ECR repository

Infrastructure as code

- ☐ 6. All infrastructure defined in Terraform — nothing created by hand
- ☐ 7. Terraform state in a remote backend with locking
- ☐ 8. No hardcoded values that should be variables
- ☐ 9. `terraform destroy` then `terraform apply` rebuilds the whole stack successfully

Network and compute

- ☐ 10. Application container runs in a **private** subnet
- ☐ 11. Only the load balancer sits in a public subnet
- ☐ 12. Security groups reference other security groups, not `0.0.0.0/0`
- ☐ 13. Health check endpoint configured and the target group is healthy
- ☐ 14. At least two Availability Zones used
- ☐ 15. Application reachable through the load balancer URL

Database and configuration

- ☐ 16. Database in a private subnet, `publicly_accessible = false`
- ☐ 17. Database password stored in Secrets Manager
- ☐ 18. Application config in Parameter Store, read at runtime
- ☐ 19. **No credentials anywhere in the repository** — verified by a scan
- ☐ 20. Encryption at rest enabled on the database and buckets
- ☐ 21. Automated backups enabled with a non-zero retention period

Frontend and serverless

- ☐ 22. Frontend served through CloudFront; S3 bucket is not public
- ☐ 23. Attachments uploaded via presigned URL, not through the API
- ☐ 24. Lambda triggered by S3 upload, working end to end

Pipeline

- ☐ 25. Push to `main` deploys automatically with no manual AWS steps
- ☐ 26. A failing test or secret-scan **blocks** the deployment
- ☐ 27. Smoke test runs against the deployed environment after deploy

Operations

- ☐ 28. Logs in CloudWatch with a finite retention period
- ☐ 29. Dashboard showing requests, errors, latency, CPU/memory
- ☐ 30. Three working alarms wired to a notification target

Housekeeping

- ☐ 31. Every resource tagged with Project, Owner, Environment, CostCenter
- ☐ 32. IAM task role scoped to specific actions and resources — no `"*"` with `"*"`
- ☐ 33. Spend within budget, with a one-page cost report
- ☐ 34. `README.md` a new joiner could follow to deploy this from scratch

34 items. If you can tick all 34, you are deployment ready. That is the whole point of these two weeks.

7. Five things that are pass/fail

These sit outside the marks. Failing any one caps your grade regardless of your score, because each one would be a serious incident on a real project.

	Requirement
1	No credentials, keys or passwords committed to the repository
2	No IAM policy with <code>"Action": "*" on "Resource": "*"</code>
3	The database is not reachable from the internet
4	The stack rebuilds from zero using documented commands
5	<code>terraform destroy</code> leaves nothing billable behind

8. How you are scored

	Criterion	Points
Individual (M0–M5)		60
	Containerisation and image hygiene	10
	Infrastructure as code and reproducibility	15
	Network and compute	15

	Criterion	Points
Pod (M6–M8)	deployment	
	Database, configuration and secrets	12
	Frontend and serverless function	8
		40
	CI/CD pipeline	15
	Observability	10
	Security basics	8
	Cost, tagging and teardown	7
	Total	100

60% of your score is individual. Your pod cannot carry you, and you cannot be dragged down by a weak pod on the part that matters most.

9. Stretch goals — only after all 34 checklist items pass

Stretch	Points
HTTPS on a real domain with an ACM certificate	+2
Auto-scaling on the ECS service, demonstrated under load	+2
Cognito login instead of the stub authentication	+2
Blue/green deployment with automatic rollback	+3
Scheduled nightly shutdown and morning start-up to cut cost	+1
Swap RDS for DynamoDB in a branch and write up what changed	+2

Capped at 100 total. **A broken stretch goal costs you more than a missing one.**

10. Honest advice from people who have marked this before

Do the milestones in order. Every fresher who jumps to the pipeline before their IaC works ends up debugging two unfamiliar things at once. It never goes well.

When something doesn't work, read the logs before changing anything. The single biggest gap between a fresher and a mid-level engineer is not knowledge — it is the instinct to look at the error before guessing. Your first move should always be: what do the ECS task logs and the stopped task reason actually say?

Type the commands. You will be tempted to copy-paste Terraform from the internet or from an AI assistant. That is allowed, and it is also how you arrive at demo day unable to explain your own infrastructure. Copy it, then retype it, then change one value and see what breaks.

Destroy and rebuild every day or two. It is the fastest way to find out that your setup secretly depends on something you clicked once and forgot. Better to discover that on Day 4 than Day 10.

Being stuck is completely normal. Ask after 45 minutes, not after two days. There are 30 minutes of facilitator office hours every day, and the whole batch benefits when you ask in the shared channel.

11. AI assistants

You may use them. But on demo day a facilitator will open your Terraform, point at a block, and ask what it does and what happens if a value changes. **Code you cannot explain scores as if you had not submitted it** — so use AI to learn faster, not to skip the learning.

Two weeks. One application. Nine AWS services. At the end of it, you will be able to say something most people finishing cloud training cannot: “I have deployed and run a real application on AWS, and I can do it again tomorrow.”

Let's get started.